

# ACID Properties

Chapter 6 | PostgreSQL Complete Guide Difficulty: Intermediate | Estimated Reading Time: 30 minutes

---

## Table of Contents

- [Learning Objectives](#)
  - [Theory: ACID Overview](#)
  - [Atomicity](#)
  - [Consistency](#)
  - [Isolation](#)
  - [Durability](#)
  - [Isolation Levels in PostgreSQL](#)
  - [MVCC: How PostgreSQL Implements ACID](#)
  - [ASCII Visual Diagrams](#)
  - [Practical Examples](#)
  - [PostgreSQL Commands](#)
  - [SQL Examples](#)
  - [Common Mistakes](#)
  - [Best Practices](#)
  - [Performance Considerations](#)
  - [Interview Questions](#)
  - [Interview Answers](#)
  - [Hands-on Exercises](#)
  - [Solutions](#)
  - [Advanced Notes](#)
  - [Cross-References](#)
- 

## Learning Objectives

By the end of this chapter, you will be able to:

- Define all four ACID properties and explain their importance in database transactions
  - Demonstrate each ACID property with concrete SQL examples
  - Understand the four transaction isolation levels and the anomalies they prevent
  - Explain how PostgreSQL implements ACID using MVCC (Multi-Version Concurrency Control)
  - Identify real-world scenarios where ACID violations would cause serious problems
- 

## Theory: ACID Overview

**ACID** is an acronym for four properties that guarantee reliable database transactions:

- **A**tomicity
- **C**onsistency
- **I**solation
- **D**urability

These properties were formalized by Andreas Reuter and Theo Härder in 1983. They are the foundation of reliable transactional databases.

## What is a Transaction?

A **transaction** is a sequence of one or more SQL operations treated as a single logical unit of work. A transaction either:

- **Commits** (all operations succeed and changes are permanent), or
- **Rolls back** (all operations are undone as if they never happened)

```
BEGIN;                -- Start transaction
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;               -- Or ROLLBACK;
```

---

## Atomicity

### Definition

A transaction is **all or nothing**. Either all operations in the transaction complete successfully, or none of them take effect.

### Why It Matters

Consider a bank transfer of \$500 from Account A to Account B:

1. Debit \$500 from Account A
2. Credit \$500 to Account B

Without atomicity: If step 1 succeeds but step 2 fails (system crash), \$500 disappears. Money is destroyed.

With atomicity: Either both steps succeed or both are rolled back. The total money is preserved.

### How PostgreSQL Implements Atomicity

- **Write-Ahead Logging (WAL):** Every change is logged before it's applied to data files
- On crash, PostgreSQL replays the WAL to recover completed transactions and discard incomplete ones
- **BEGIN/COMMIT/ROLLBACK:** Explicit transaction boundaries

### Example

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;
-- Simulated failure here
ROLLBACK; -- Both operations are undone; Account 1 retains its original balance
```

---

## Consistency

### Definition

A transaction brings the database from one **valid state** to another **valid state**. The database must satisfy all defined rules (constraints, triggers, cascades) before and after the transaction.

### Note on ACID-Consistency vs. CAP-Consistency

ACID-C = database integrity constraints are never violated CAP-C = all distributed nodes see the same data

These are different concepts. This chapter covers ACID-Consistency.

### Types of Consistency Rules

- 1. **Explicit constraints:** NOT NULL, UNIQUE, CHECK, FOREIGN KEY
- 2. **Triggers:** Custom logic enforced on data changes
- 3. **Application-level invariants:** Business rules (e.g., account balance cannot go negative)

### Example

```
-- Constraint: balance cannot be negative
ALTER TABLE accounts ADD CONSTRAINT chk_balance CHECK (balance >= 0);

BEGIN;
UPDATE accounts SET balance = balance - 10000
WHERE account_id = 1 AND balance >= 10000; -- Only debit if sufficient funds
COMMIT;

-- If balance would go negative, the CHECK constraint prevents the transaction
```

## Isolation

### Definition

Concurrent transactions execute as if they were **serial** — each transaction is isolated from the effects of other in-progress transactions.

### Read Phenomena (Anomalies Without Isolation)

| Anomaly               | Description                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------------------|
| Dirty Read            | Transaction T2 reads data written by T1 that has NOT yet committed. If T1 rolls back, T2 has read invalid data |
| Non-Repeatable Read   | T1 reads the same row twice; T2 modifies that row between T1's reads. T1 gets different values                 |
| Phantom Read          | T1 executes the same range query twice; T2 inserts/deletes rows matching that range between T1's queries       |
| Serialization Anomaly | The combined result of concurrent transactions is impossible to achieve by any serial execution order          |

## Durability

### Definition

Once a transaction is **committed**, it is permanently recorded and will survive system failures (crashes, power outages).

## How PostgreSQL Implements Durability

1. **WAL (Write-Ahead Log):** Changes are written to the WAL log before data files
2. **fsync:** PostgreSQL calls `fsync()` to flush WAL to disk before acknowledging COMMIT
3. **Checkpoint:** Periodically writes dirty pages from memory to disk
4. **Crash Recovery:** On restart, PostgreSQL replays unconfirmed WAL entries

## Performance Trade-off

```
-- Default: durable (fsync to disk on each commit)
SET synchronous_commit = on;

-- Faster but risks last ~200ms of transactions on crash
SET synchronous_commit = off;

-- Write to OS buffer only (risky)
SET synchronous_commit = local;
```

## Isolation Levels in PostgreSQL

PostgreSQL implements four SQL standard isolation levels:

| Level            | Dirty Read   | Non-Repeatable Read | Phantom Read      | Serialization Anomaly |
|------------------|--------------|---------------------|-------------------|-----------------------|
| READ UNCOMMITTED | Possible     | Possible            | Possible          | Possible              |
| READ COMMITTED   | Not possible | Possible            | Possible          | Possible              |
| REPEATABLE READ  | Not possible | Not possible        | Not possible (PG) | Possible              |
| SERIALIZABLE     | Not possible | Not possible        | Not possible      | Not possible          |

**Note:** PostgreSQL's `READ UNCOMMITTED` behaves like `READ COMMITTED` — PostgreSQL never allows dirty reads.

## Setting Isolation Level

```
-- For a specific transaction
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- For the session
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- For the database
ALTER DATABASE mydb SET DEFAULT_TRANSACTION_ISOLATION TO 'serializable';
```

---

## MVCC: How PostgreSQL Implements ACID

PostgreSQL uses **Multi-Version Concurrency Control (MVCC)** to implement isolation without locking readers.

### How MVCC Works

Instead of overwriting data in place, PostgreSQL keeps multiple versions of each row:

- Every row has hidden columns: `xmin` (transaction that created it) and `xmax` (transaction that deleted/updated it)
- A reader sees the version of a row that was visible at the start of its transaction
- A writer creates a new version of the row
- Old versions are cleaned up by `VACUUM`

### Benefits

- **Readers never block writers**
  - **Writers never block readers**
  - High concurrency with strong isolation guarantees
- 

## ASCII Visual Diagrams

### Diagram 1: Atomicity — All or Nothing

```
ATOMICITY
=====

SCENARIO: Transfer $500 from Alice to Bob

WITHOUT ATOMICITY:          WITH ATOMICITY:
=====                     =====
Step 1: Debit Alice -$500    BEGIN;
  Alice: $1000 --> $500 OK    Step 1: Debit Alice -$500
Step 2: Credit Bob +$500      Alice: $1000 --> $500
  SYSTEM CRASH!              Step 2: Credit Bob +$500
                             SYSTEM CRASH!  X
                             ROLLBACK (on recovery)
  Alice: $500                 Alice: $1000 (restored)
  Bob:  $1000 (unchanged)      Bob:  $1000 (unchanged)
  $500 DISAPPEARED!           MONEY PRESERVED!
```

### Diagram 2: Isolation Anomalies

```
DIRTY READ (prevented at READ COMMITTED+)
=====
T1: UPDATE accounts SET bal=0 WHERE id=1
    |
T2: | SELECT bal FROM accounts WHERE id=1
    | Returns 0 (T1 not yet committed!)
    |
```

T1: ROLLBACK

T2 read a value that NEVER EXISTED in committed state!

NON-REPEATABLE READ (prevented at REPEATABLE READ+)

=====

T1: SELECT bal FROM accounts WHERE id=1 --> 1000

|     |  |                                        |
|-----|--|----------------------------------------|
|     |  |                                        |
| T2: |  | UPDATE accounts SET bal=500 WHERE id=1 |
|     |  | COMMIT                                 |
|     |  |                                        |

T1: SELECT bal FROM accounts WHERE id=1 --> 500 (CHANGED!)

T1 sees different values for the same row within one transaction.

PHANTOM READ (prevented at SERIALIZABLE)

=====

T1: SELECT COUNT(\*) FROM orders WHERE total > 100 --> 5

|     |  |                                         |
|-----|--|-----------------------------------------|
|     |  |                                         |
| T2: |  | INSERT INTO orders (total) VALUES (150) |
|     |  | COMMIT                                  |
|     |  |                                         |

T1: SELECT COUNT(\*) FROM orders WHERE total > 100 --> 6 (NEW ROW!)

T1 sees a new row that "appeared" (a phantom).

### Diagram 3: MVCC Row Versions

MVCC VERSIONS IN PostgreSQL

=====

Time -->

|                            |                        |
|----------------------------|------------------------|
| T10 (Alice updates salary) | T15 (Bob reads salary) |
|----------------------------|------------------------|

Row in heap:

|                 |                           |
|-----------------|---------------------------|
| +-----+-----+   |                           |
| xmin=5, xmax=10 | xmin=10, xmax=0 (current) |
| salary = 50000  | salary = 60000            |
| +-----+-----+   |                           |
| Deleted by T10  | Created by T10            |

Bob's snapshot (started before T10 committed):

Sees xmin=5, xmax=10 version --> salary = 50000

Carol's snapshot (started after T10 committed):

Sees xmin=10, xmax=0 version --> salary = 60000

Neither read blocked the write. No locks needed!

---

## Practical Examples

## Example 1: Financial Transfer

Bank transfers are the canonical ACID example. All four properties are critical:

- **A:** Debit and credit are atomic
- **C:** No account goes negative, total money is preserved
- **I:** Concurrent transfers don't interfere
- **D:** Committed transfers survive power outages

## Example 2: Ticket Booking

Concert ticket booking:

- **A:** Reserve seat and charge card atomically
- **C:** Seat capacity constraint never exceeded
- **I:** Two users can't book the same seat (isolation prevents double-booking)
- **D:** Confirmed bookings persist

## Example 3: Inventory Management

E-commerce inventory:

- **A:** Order placement and inventory decrement are one transaction
- **C:** Stock quantity cannot go below 0
- **I:** Two orders for the last item — only one should succeed
- **D:** Once order is confirmed, it persists

---

## PostgreSQL Commands

```
-- View current transaction isolation level
SHOW transaction_isolation;

-- View all MVCC-related row info
SELECT xmin, xmax, ctid, * FROM accounts LIMIT 5;

-- Check for long-running transactions (blocks VACUUM)
SELECT
    pid,
    now() - xact_start AS duration,
    query,
    state
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY duration DESC;

-- Check for locks
SELECT
    pid,
    locktype,
    relation::regclass,
    mode,
    granted
FROM pg_locks
```

```
WHERE NOT granted;

-- Check for deadlocks in pg_log
-- grep "deadlock" postgresql.log
```

---

## SQL Examples

### Example 1: Basic Transaction

```
BEGIN;
    UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;
    UPDATE accounts SET balance = balance + 1000 WHERE account_id = 2;
COMMIT;
```

### Example 2: Transaction with Error Handling (SAVEPOINT)

```
BEGIN;
    INSERT INTO orders (customer_id, total) VALUES (1, 500.00);

    SAVEPOINT after_order; -- Create a savepoint

    INSERT INTO order_items (order_id, product_id, qty) VALUES (1, 99, 1);
    -- If this fails (product doesn't exist), rollback to savepoint only

    -- If product doesn't exist:
    ROLLBACK TO SAVEPOINT after_order;
    -- Order still exists, but no order items

COMMIT;
```

### Example 3: Demonstrating READ COMMITTED

```
-- Session 1
BEGIN;
UPDATE products SET price = 9999 WHERE product_id = 1;
-- NOT YET COMMITTED

-- Session 2 (READ COMMITTED is default)
BEGIN;
SELECT price FROM products WHERE product_id = 1;
-- Returns original price (not 9999) -- NO DIRTY READ
COMMIT;

-- Session 1
COMMIT;

-- Session 2 (new read after Session 1 committed)
```

```
SELECT price FROM products WHERE product_id = 1;
-- Now returns 9999
```

#### Example 4: REPEATABLE READ Isolation

```
-- Session 1
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT price FROM products WHERE product_id = 1; -- Returns 100

-- Session 2
BEGIN;
UPDATE products SET price = 200 WHERE product_id = 1;
COMMIT;

-- Session 1 (still in its transaction)
SELECT price FROM products WHERE product_id = 1; -- STILL Returns 100 (snapshot!)
COMMIT;
```

#### Example 5: SERIALIZABLE Isolation

```
-- Prevents all anomalies, including write skew
-- Session 1
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors_on_call WHERE day = 'Monday'; -- Returns 2

-- Session 2
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM doctors_on_call WHERE day = 'Monday'; -- Returns 2

-- Both decide to remove themselves (each sees 2, thinks 1 will remain)
-- Session 1
DELETE FROM doctors_on_call WHERE doctor_id = 1 AND day = 'Monday';
COMMIT; -- Succeeds

-- Session 2
DELETE FROM doctors_on_call WHERE doctor_id = 2 AND day = 'Monday';
COMMIT; -- ERROR! Serialization failure detected -- protects against 0 doctors
```

#### Example 6: Row-Level Locking (SELECT FOR UPDATE)

```
-- Lock a row for update (prevents concurrent modifications)
BEGIN;
SELECT * FROM inventory
WHERE product_id = 1
FOR UPDATE; -- Acquires row-level lock

-- Now update safely
```

```
UPDATE inventory SET qty = qty - 1 WHERE product_id = 1;
COMMIT; -- Lock released
```

### Example 7: SKIP LOCKED (Queue Pattern)

```
-- Process jobs without blocking (useful for job queues)
BEGIN;
SELECT job_id, payload
FROM job_queue
WHERE status = 'pending'
ORDER BY created_at
LIMIT 1
FOR UPDATE SKIP LOCKED; -- Skip rows locked by other workers

-- Process the job
UPDATE job_queue SET status = 'processing' WHERE job_id = ?;
COMMIT;
```

### Example 8: NOWAIT (Fail Fast on Lock)

```
-- Don't wait for locked rows; fail immediately
BEGIN;
SELECT * FROM accounts
WHERE account_id = 1
FOR UPDATE NOWAIT; -- Raises ERROR if row is locked
-- Useful for user-facing operations where waiting is unacceptable
COMMIT;
```

### Example 9: Checking Transaction State

```
-- Is the current transaction in an error state?
SELECT
    txid_current(),
    pg_current_xact_id_if_assigned() IS NOT NULL AS in_transaction;

-- View transaction IDs
SELECT xmin, xmax, ctid, account_id, balance
FROM accounts;
```

### Example 10: Deferred Constraint Checking

```
-- Sometimes you need to temporarily violate a constraint within a transaction
CREATE TABLE circular_refs (
    id INT PRIMARY KEY,
    parent_id INT REFERENCES circular_refs(id) DEFERRABLE INITIALLY DEFERRED
);
```

```
BEGIN;
INSERT INTO circular_refs VALUES (1, 2); -- parent 2 doesn't exist yet
INSERT INTO circular_refs VALUES (2, 1); -- creates the cycle
COMMIT; -- Constraint checked here -- both rows exist, constraint satisfied
```

### Example 11: Advisory Locks (Application-Level)

```
-- Acquire an application-level lock (not tied to a row)
SELECT pg_advisory_lock(12345); -- Lock by integer key
-- ... do some work ...
SELECT pg_advisory_unlock(12345);

-- Try to acquire without waiting
SELECT pg_try_advisory_lock(12345); -- Returns TRUE/FALSE
```

### Example 12: WAL Configuration Check

```
-- Check WAL settings (durability-related)
SHOW wal_level;
SHOW fsync;
SHOW synchronous_commit;
SHOW checkpoint_completion_target;
SHOW max_wal_size;
```

---

## Common Mistakes

### Mistake 1: Long-Running Transactions

Keeping transactions open for a long time blocks VACUUM (which reclaims dead row versions), leading to table bloat.

```
-- BAD: Transaction held open during user interaction
BEGIN;
-- ... wait for user input for 30 seconds ...
UPDATE orders SET status = 'approved' WHERE order_id = 1;
COMMIT;

-- GOOD: Keep transactions short
-- Do all business logic OUTSIDE the transaction
-- Open transaction only for the actual database operations
```

### Mistake 2: Not Using Transactions for Multi-Step Operations

```
-- BAD: Steps 1 and 2 are separate auto-commit transactions
INSERT INTO orders (customer_id, total) VALUES (1, 100);
UPDATE inventory SET qty = qty - 1 WHERE product_id = 5;
-- If second statement fails, order exists but inventory not decremented
```

```
-- GOOD: Wrap in a transaction
BEGIN;
  INSERT INTO orders (customer_id, total) VALUES (1, 100);
  UPDATE inventory SET qty = qty - 1 WHERE product_id = 5;
COMMIT;
```

### Mistake 3: Using Low Isolation Levels for Critical Operations

READ COMMITTED is the default and appropriate for most operations. But for financial operations (e.g., account balance checks + transfers), use REPEATABLE READ or SERIALIZABLE.

### Mistake 4: Ignoring Lock Contention

High-concurrency systems need careful lock management. `SELECT FOR UPDATE` on a hot row creates a lock bottleneck.

### Mistake 5: Disabling fsync for Performance

```
# NEVER do this in production
fsync = off # Disables durability! Data corruption on crash guaranteed
```

### Mistake 6: Not Handling Serialization Failures

With SERIALIZABLE isolation, transactions can fail with `ERROR: could not serialize access due to concurrent update`. Applications must be designed to retry these transactions.

---

## Best Practices

1. **Keep transactions short:** Open late, close early. Do calculations and validations outside the transaction.
2. **Use appropriate isolation levels:** Default READ COMMITTED for most cases; SERIALIZABLE for complex concurrency requirements.
3. **Always handle ROLLBACK :** Use proper error handling in your application code to roll back on failure.
4. **Use SAVEPOINTS for partial rollbacks:** In long multi-step transactions, use savepoints to recover from partial failures.
5. **Monitor long-running transactions:** Alert when a transaction exceeds 60 seconds.
6. **Use SELECT FOR UPDATE for pessimistic locking:** When you must prevent concurrent modification of a row.
7. **Use ON CONFLICT for optimistic concurrency:** Prefer `INSERT ... ON CONFLICT` over checking existence first, then inserting.

---

## Performance Considerations

- **MVCC bloat:** Every UPDATE creates a new row version. Old versions accumulate until VACUUM removes them. Monitor table bloat with `pgstattuple` .
- **VACUUM and AUTOVACUUM:** Essential for reclaiming dead tuples. Never disable autovacuum.
- **Index bloat:** Indexes also accumulate dead entries. Use `REINDEX CONCURRENTLY` for maintenance.
- **Lock wait timeout:** Set `lock_timeout` to prevent transactions from waiting indefinitely.
- **Statement timeout:** Set `statement_timeout` to kill long-running queries.

```
-- Protect against long waits
SET lock_timeout = '5s';
SET statement_timeout = '30s';

-- Check table bloat
SELECT
    schemaname,
    tablename,
    n_dead_tup,
    n_live_tup,
    ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

## Interview Questions

1. What does ACID stand for?
2. Explain atomicity with a real-world example.
3. What is the difference between ACID-Consistency and CAP-Consistency?
4. What are the four SQL isolation levels?
5. What is a dirty read? At what isolation level is it prevented?
6. What is a phantom read? How does PostgreSQL prevent it?
7. What is MVCC and what problem does it solve?
8. What is the difference between ROLLBACK and ROLLBACK TO SAVEPOINT?
9. What is `SELECT FOR UPDATE` used for?
10. Why is it dangerous to have long-running transactions in PostgreSQL?

## Interview Answers

### Q4: Four Isolation Levels

1. **READ UNCOMMITTED** — Can read uncommitted data (dirty reads). PostgreSQL treats this as READ COMMITTED.
2. **READ COMMITTED** — Only reads committed data; default in PostgreSQL. Prevents dirty reads.
3. **REPEATABLE READ** — Snapshot taken at start of transaction; prevents non-repeatable reads and phantoms (in PostgreSQL).
4. **SERIALIZABLE** — Full isolation; transactions appear serial. Prevents all anomalies including write skew.

### Q7: MVCC

Multi-Version Concurrency Control keeps multiple versions of each row. Each transaction sees a snapshot of the database from the moment it started. Readers never block writers (they read the old version), and writers

never block readers (they create new versions). This provides high concurrency without sacrificing isolation. Old versions are reclaimed by VACUUM.

### Q10: Long-Running Transactions

Long transactions:

1. Prevent VACUUM from removing old row versions (the transaction might need them), causing table bloat
  2. Hold locks, blocking other transactions
  3. Increase recovery time on crash (WAL must be replayed further back)
  4. Can cause `xid wraparound` — a catastrophic situation where PostgreSQL runs out of transaction IDs
- 

## Hands-on Exercises

### Exercise 1: Atomicity Test

Write a transaction that transfers money between accounts. Verify that if you ROLLBACK, the original balances are restored.

### Exercise 2: Isolation Level Comparison

Using two psql sessions, demonstrate a non-repeatable read at READ COMMITTED isolation level, then show it's prevented at REPEATABLE READ.

### Exercise 3: Deadlock Creation and Detection

Create a deadlock between two sessions. Observe how PostgreSQL detects and resolves it.

### Exercise 4: SAVEPOINT Usage

Write a transaction with a savepoint that processes multiple orders. If one order fails (bad product\_id), roll back only that order but commit the others.

### Exercise 5: Monitoring Locks

Use `pg_locks` and `pg_stat_activity` to identify which session is blocking another.

---

## Solutions

### Solution 1

```
CREATE TABLE accounts (  
  account_id SERIAL PRIMARY KEY,  
  name        VARCHAR(100),  
  balance     DECIMAL(12,2) CHECK (balance >= 0)  
);  
INSERT INTO accounts (name, balance) VALUES ('Alice', 1000), ('Bob', 500);  
  
BEGIN;  
UPDATE accounts SET balance = balance - 300 WHERE account_id = 1;  
UPDATE accounts SET balance = balance + 300 WHERE account_id = 2;
```

```
-- Check intermediate state (within transaction)
SELECT * FROM accounts;
ROLLBACK; -- Undo both changes
-- Verify balances are original
SELECT * FROM accounts; -- Alice: 1000, Bob: 500
```

### Solution 3 (Deadlock)

```
-- Session 1
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1; -- Locks row 1

-- Session 2
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE account_id = 2; -- Locks row 2

-- Session 1 (now tries to lock row 2 - WAITS)
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;

-- Session 2 (now tries to lock row 1 - DEADLOCK DETECTED)
UPDATE accounts SET balance = balance + 100 WHERE account_id = 1;
-- PostgreSQL kills one transaction: ERROR: deadlock detected
```

---

## Advanced Notes

### Two-Phase Locking (2PL) vs. MVCC

Traditional databases use **2PL** — readers and writers both acquire locks, causing contention. PostgreSQL uses **MVCC** — readers never need locks, dramatically improving concurrency for read-heavy workloads.

### Predicate Locks (SSI)

Serializable Snapshot Isolation (SSI) in PostgreSQL uses "predicate locks" — not actual row locks, but tracking of which data ranges a transaction has read. This enables detecting write skew without requiring exclusive locks. It's the mechanism behind `SERIALIZABLE` isolation in PostgreSQL.

### XID Wraparound

PostgreSQL uses 32-bit transaction IDs (XIDs). After ~2 billion transactions, XIDs wrap around. PostgreSQL uses a "vacuum freeze" process to mark old rows as permanently visible, preventing catastrophic data loss from wraparound. Monitor `pg_stat_user_tables.n_dead_tup` and `age(datfrozenxid)`.

---

## Cross-References

- **Previous:** [05 - CAP Theorem](#)
- **Next:** [07 - BASE Properties](#)
- **Related:** [03 - Relational Databases](#) — Constraints that enforce C
- **See Also:** [04 Advanced SQL/08 transactions.md](#) — Advanced transaction patterns
- **See Also:** [07 Performance/02 locking.md](#) — Lock management and contention
- **See Also:** [09 Administration/05 vacuum.md](#) — VACUUM and MVCC maintenance

